LAB 1:

Date: 17 de mayo de 2025

Professor:

Dr. Francisco Javier Rodríguez Navarrete

Students:

Alonso Emmanuel López Macias

Cesar Eduardo Inda Ceniceros.

Team: 14

Introduction

In this lab, a sequential digital counter is developed in Verilog, capable of counting from 0 to 9999, both upwards and downwards. This type of circuit is fundamental in digital systems, as it allows controlling events, measuring time, generating sequences, and addressing memories. The project includes parameterizing the counter, integrating it with an FPGA, displaying the output on 7-segment displays, and simulating the system's behavior through a testbench.

Requirements

- Design a counter that can reach the decimal value 9999.
- The counter must count up or down based on the up_down signal.
- The initial value can be loaded via the load signal and the input data_in.
- If the loaded value exceeds 9999, the counter resets to 0.
- The output q must be displayed on 4 seven-segment displays.
- When the counter value is 9999, LEDG8 should turn on.
- When the counter value is 8, LEDG8 should turn off.
- The clk is generated by a button (KEY0) and reset is triggered by KEY1.
- The system must be verified using a testbench in simulation.

Development

The design was expanded to 14 bits to support counting up to 9999. The module includes an up_down signal to determine the counting direction, an en signal to enable counting, and a load signal to initialize the counter with an external value (data_in).

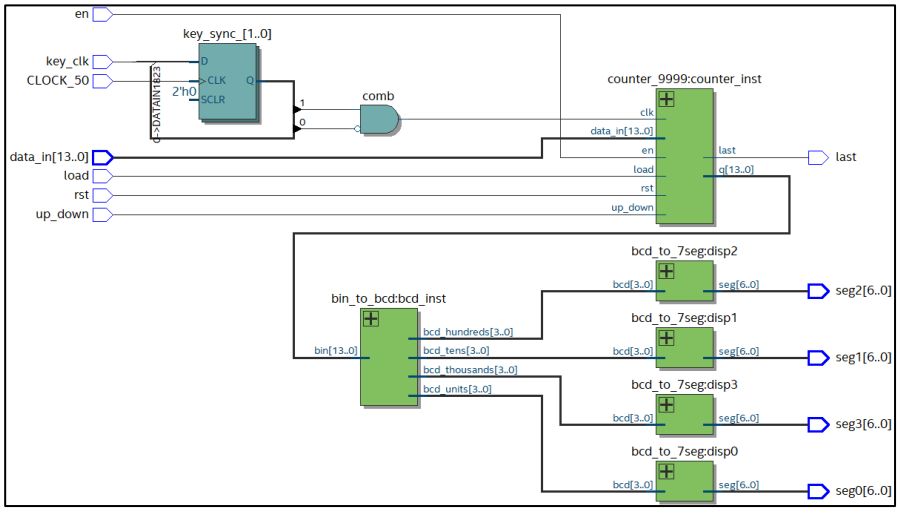

Figure: 1 RTL Counter9999

The consideration logic for the counter test section on the testbench is:

```
/*This stage is for test the counter in test-bench section across the
   the main principle in the stage the limit preserv the section "limit"*/

module counter_9999 #(
    parameter WIDTH = 14,
    parameter LIMIT = 9999
)(
    input wire clk,
    input wire rst,
    input wire en,
    input wire load,
    input wire up_down,  // 0 = up, 1 = down
    input wire [WIDTH-1:0] data_in,
    output wire last,
    output reg [WIDTH-1:0] q
);

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            q <= 0;
        end else if (en) begin
            if (load) begin
                if (data_in > LIMIT)
                    q <= 0;
                else
                    q <= data_in;
            end else if (!up_down) begin
                if (q == LIMIT)
                    q <= 0;
                else
                    q <= q + 1;
            end else begin
                if (q == 0)
                    q <= LIMIT;
                else
                    q <= q - 1;
            end
        end
    end

    assign last = (q == LIMIT || q == 0);

endmodule
```

Figure: 2 Counter_9999.v

The consideration for the top counter display is as following but, in this we add the section and considerations for bin converter to bcd and bcd to seven segment converter this logic is part from de other labs done before and is a recycled logic, is important know that in this kind of integrations exist some variants for consider in the around stages and the careful the main logic section in the set entity member of the project.

This represents the logic for set – top entity in the project with the other converter functions call backs.

```
module top_counter_display (
    input CLOCK_50,           // MainClock_50MHZ
    input key_clk,            // KEY[0] --> Counter++ || Counter+1
    input rst,
    input en,
    input load,
    input up_down,
    input [13:0] data_in,
    output last,
    output [6:0] seg0, seg1, seg2, seg3
);

    // === Sincronizador y detector de flanco de bajada ===
    reg key_sync_0, key_sync_1;
    always @(posedge CLOCK_50) begin
        key_sync_0 <= key_clk;
        key_sync_1 <= key_sync_0;
    end

    wire clk_pulse = key_sync_1 & ~key_sync_0;

    wire [13:0] q;
    wire [3:0] u, t, h, th;

    counter_9999 counter_inst (
        .clk(clk_pulse),       // Pulso generado al presionar el botón
        .rst(rst),
        .en(en),
        .load(load),
        .up_down(up_down),
        .data_in(data_in),
        .last(last),
        .q(q)
    );

    bin_to_bcd bcd_inst (
        .bin(q),
        .bcd_units(u),
        .bcd_tens(t),
        .bcd_hundreds(h),
        .bcd_thousands(th)
    );

    bcd_to_7seg disp0 (.bcd(u),  .seg(seg0));
    bcd_to_7seg disp1 (.bcd(t),  .seg(seg1));
    bcd_to_7seg disp2 (.bcd(h),  .seg(seg2));
    bcd_to_7seg disp3 (.bcd(th), .seg(seg3));

endmodule
```

Figure: 3 top_counter_display. v

Testbench

This is the consideration for the logic in testbench structure and then how to apply to use in the section Quartus – ModelSim – Altera.

```
`timescale 1ns / 1ps

module tb_top_counter_display();

    reg key_clk;
    reg rst;
    reg en;
    reg load;
    reg up_down;
    reg [13:0] data_in;
    wire last;
    wire [6:0] seg0, seg1, seg2, seg3;

    // Instancia del módulo principal
    top_counter_display uut (
        .key_clk(key_clk),
        .rst(rst),
        .en(en),
        .load(load),
        .up_down(up_down),
        .data_in(data_in),
        .last(last),
        .seg0(seg0),
        .seg1(seg1),
        .seg2(seg2),
        .seg3(seg3)
    );

    // Tarea para simular flanco de bajada del botón
    task clk_pulse;
    begin
        key_clk = 1;
        #5;
        key_clk = 0;  // Flanco de bajada (activo)
        #5;
    end
    endtask
```

Figure: 4 testbench_part1

After this we can add the considerations for each one condition in clock pulsations and the stored values in the other variables, we can observe that in this section, we defined a overflow section, underflow, and the enable and disable conditions.

The second section of the testbench code:

```
    initial begin
        // Inicialización
        key_clk = 1; // botón "no presionado"
        rst = 1; en = 0; load = 0; up_down = 0; data_in
        #10;

        // Liberar reset
        rst = 0;
        #10;

        // Cargar valor válido (ej. 20)
        en = 1;
        data_in = 14'd20;
        load = 1;
        clk_pulse();    // Pulso para cargar
        load = 0;

        // Contar hacia arriba
        up_down = 0;
        repeat (5) clk_pulse();

        // Contar hacia abajo
        up_down = 1;
        repeat (5) clk_pulse();

        // Cargar valor inválido (>9999)
        data_in = 14'd12000;
        load = 1;
        clk_pulse();
        load = 0;

        // Contar desde 9999 para probar desbordamiento
        data_in = 14'd9999;
        load = 1;
        clk_pulse();
        load = 0;
        up_down = 0;
        clk_pulse(); // Debería volver a 0

        // Contar hacia abajo desde 0 para underflow
        data_in = 14'd0;
        load = 1;
        clk_pulse();
        load = 0;
        up_down = 1;
        clk_pulse(); // Debería ir a 9999

        // Deshabilitar enable
        en = 0;
        repeat (3) clk_pulse(); // No debería cambiar

        $stop;
    end
endmodule
```

Figure: 5 Testbench_part2

If possible, add more conditions if is necessary for validate other limit count or if not can resolve a complete reset, however we need to know the main Requeriments for each condition for complete.

After run the simulation in ModelSim- Altera we can observe as following:

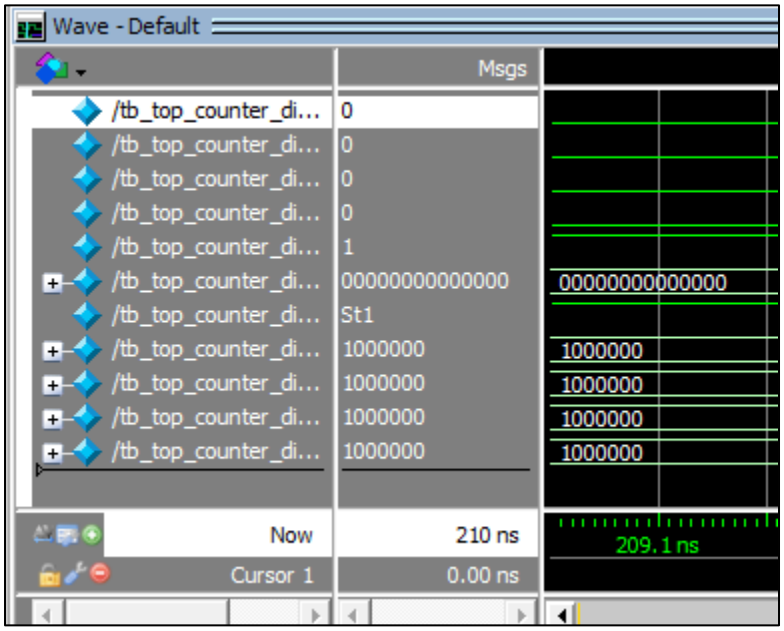

Figure: 6 Testbench_counter9999

FPGA Implementation

Flow Status	Successful - Sun Jun 01 10:31:10 2025
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	counter_9999
Top-level Entity Name	top_counter_display
Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	199 / 114,480 (< 1 %)
Total registers	16
Total pins	49 / 529 (9 %)
Total virtual pins	0
Total memory bits	0 / 3,981,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 532 (0 %)
Total PLLs	0 / 4 (0 %)

Figure: 7 Compile Report

GitHub Repository

https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA_3/tree/Lab1_Counter9999

Conclusion/Issues

Cesar Eduardo Inda Ceniceros

Designing a parameterized counter in Verilog allows adapting the circuit to different counting ranges and facilitates reuse in future projects. Moreover, integrating it with displays and output elements improves visual interpretation of results on an FPGA.

Alonso Emmanuel López Macias.
Simulation through a testbench with controlled pulses allows accurate validation of the circuit's behavior under conditions similar to the physical environment, helping detect logic errors before hardware implementation.